

## רמזים

## שים לב

שלוש דרגות בכל קושי. קרא את הדחיפה, נסה עשר דקות, ורק אז את האסטרטגיה. הכמעט-פתרון הוא הרשת, לא הקיצור. שני הכלים שפותרים כמעט הכול כאן: DevTools > Application > Local Storage, שבו רואים את הערך המדויק ואפשר לערוך אותו ביד; ו-DevTools > Elements, על שורת עיר – ה- data-state שם אומר לך מה היישום חושב שקורה.

## א · "העמוד ריק והכול idle, למרות שרעננתי"

דחיפה. פתח Application ותסתכל אם המפתח בכלל קיים. שתי אפשרויות, ולוקח עשר שניות להפריד ביניהן: או שאף אחד לא כתב, או שאף אחד לא קרא.

אסטרטגיה. אם המפתח לא שם – `writeOne` לא הגיעה לסוף, או ש-`available` שקרית. אם המפתח שם ובתוכו נתונים – `readAll` מחזירה `{}`, ואז או ש-`migrate` דוחה את מה שהיא קיבלה, או ש-`isEntry` דוחה כל רשומה. כמעט-פתרון. הדפס `console.log(JSON.parse(localStorage.getItem(KEY)))` בקונסולה והשווה שדה-שדה עם מה ש-`isEntry` דורש. הכי נפוץ: `fetchAt` שהוא מחרוזת ולא מספר, כי חתמת `new Date().toISOString()` במקום `Date.now()` – ואז `Number.isFinite` מחזירה `false` וכל רשומה נזרקת בשקט.

ב · "כתבתי `new Date()` ואחרי רענון הכול נשבר"

דחיפה. מה `JSON.stringify(new Date())` מייצר? ומה `JSON.parse` מחזיר מזה? אסטרטגיה. מחרוזת ISO, ואז מחרוזת. `Date` לא שורד את המסע – הוא הופך למחרוזת ואין דרך חזרה בלי `reviver`. אז היישום עובד מצוין עד הרענון הראשון, ואז `getTime is not a function`. נזרק מתוך `render`, שהוא המקום האחרון שמישהו מחפש בו באג של אחסון. כמעט-פתרון. `Date.now()`. מספר. שורד את המסע כמספר, מחסיר בלי טקסים, ואין מה לשכות. זה שבוע 10, מטלה אחת אחר כך, ובשדה אחר.

## ג · "freshness מחזירה NaN או תמיד stale"

דחיפה. מה `entry.fetchAt - now` כשאחד מהם מחרוזת? ומה כש-`entry` הוא `undefined`? אסטרטגיה. שלושה מקרים ולא שניים: אין רשומה בכלל, יש רשומה בלי `fetchAt` תקין, ויש רשומה עם. שני הראשונים הם `absent`, והם צריכים לצאת מהפונקציה לפני שמישהו מחסיר משהו. כמעט-פתרון. ואז המקרה שרוב הפתרונות מפספסים: רשומה חתומה בעתיד, כי מישהו הזיז את השעון אחורה. `ageMs` שלילי הוא לא שגיאה בקוד שלך – זה נתון שכתבנו ואיננו יודעים בן כמה הוא. `Math.max(0, ageMs)` ו-`fresh`. `NaN` במקום מספר דקות על המסך – זו כן שגיאה.

## ד · "הבודק אומר ש- freshness לא נכונה, אבל אצלי היא עובדת"

**דחיפה.** קרא לה בקונסולה: `freshness({ fetchedAt: Date.now() - 20*60*1000 }, Date.now())`. מה חוזר?

**אסטרטגיה.** אם קראת ל- `Date.now()` בתוך הפונקציה במקום להשתמש בפרמטר `now`, היא כבר לא פונקציה של שני מספרים — היא פונקציה של מספר אחד ושל השעון. אי אפשר לבדוק אותה, לא על ידי הבודק ולא על ידך.

**כמעט-פתרון.** החתימה היא `freshness(entry, now = Date.now())`. ברירת המחדל היא נוחות לקוראים; בתוך הגוף משתמשים ב- `now`, תמיד. זו הדרישה בבריק, והיא מסומנת שם פעמיים.

## ה · "404 מציג לי מדידה במקום שגיאה"

**דחיפה.** `fetch` לא זרק. הוא לא זרק על 404.

**אסטרטגיה.** ההבטחה של `fetch` נדחית במקרה אחד: הבקשה לא הושלמה. 404 ו-500 הן תשובות מוצלחות של HTTP — השרת קיבל, עיבד וענה "לא". `res.ok` הוא הבדיקה, והיא צריכה להיות **בשורה שאחרי ה-`await`** ולא בסוף הפונקציה.

**כמעט-פתרון.** ואם אתה מקבל מדידה **מוזרה** במקום שגיאה — `toReading` שלך מקבל את מסמך השגיאה ומוצא בו `undefined`, ו-`undefined` מגיע אל `Math.round` שמחזיר `NaN`. תוסיף `Number.isFinite` על שלושת השדות ותקבל שגיאה במקום `NaN`.

## ו · "Promise.all נופל וכלום לא מתעדכן"

**דחיפה.** עיר אחת נכשלה. מה קרה לחמש שהצליחו?

**אסטרטגיה.** `Promise.all` נדחית עם **הסיבה הראשונה**, ואין בחוזה שלה מקום לשאר. כאן זה גרוע במיוחד: החמש כבר נכתבו לדיסק ול-`state` בזמן שהיא נדחתה, אז הדחייה לא זורקת נתונים — היא זורקת את **הכפתור**, שנשאר מושבת כי השורה שמדליקה אותו בחזרה לא רצה.

**כמעט-פתרון.** `Promise.allSettled`. אין לה מסלול כישלון בכלל: היא ממתינה לכולן, מחזירה פסק דין לכל אחת, ואין לה שום דבר שיכול להגיע ל- `catch`. השורות כבר אומרות בעצמן מה קרה להן, אז אין מה לעשות עם התוצאות מלבד לא להתעלם מהן.

## ז · "לחצתי 'רענן הכול' פעמיים והכול האדים"

**דחיפה.** מה קורה ל-`promise` של בקשה שביטלת?

**אסטרטגיה.** היא **נדחית**, עם `DOMException` בשם `AbortError`, והיא נוחתת ב- `catch` שלך בדיוק כמו כישלון אמיתי. הלחיצה השנייה ביטלה שש בקשות, אז קיבלת שש "שגיאות" שאתה יצרת.

**כמעט-פתרון.** השורה הראשונה ב- `catch`, לפני כל דבר אחר:

```
if (error.name === 'AbortError') return;
```

ולפי שם ולא לפי מחלקה: `TimeoutError` הוא גם `DOMException`, והוא כן שגיאה שצריך להראות.

## ח · "השורה מציגה שגיאה ואיבדה את המספר"

**דחיפה.** כשהניסיון נכשל — האם מחקת את מה שהיה במטמון, או רק סימנת שנכשל?

**אסטרטגיה.** אלה שני דברים שונים לגמרי. `state.cache[cityId]` הוא **מה שנמדד**; `state.status[cityId]` הוא **מה קורה עכשיו**. כישלון משנה את השני ולא נוגע בראשון.

**כמעט-פתרון.** `viewOf` מחזיר את `entry` גם במצב `error`, והשורה מציירת אותו אם הוא קיים — **עם התווית "לא מעודכן"**. "לא הצלחנו להגיע לשירות, וזה מה שהוא אמר לפני אחת עשרה דקות" הוא הרבה יותר שימושי מכרטיס ריק, כל עוד **שני החצאים** נאמרים. מספר ישן בלי תווית הוא שקר.

## ט · "ספינר ולא נמדד" מוצגים ביחד

**דחיפה.** כמה שדות מחליטים איזה מצב מוצג? אם התשובה גדולה מאחד — מצאת.

**אסטרטגיה.** זה בדיוק מה ששבוע 11 מדד. חמישה ערכים בשדה אחד לא יכולים לסתור; חמישה בוליאנים הם שלושים ושניים צירופים שרק חמישה מהם אומרים משהו.

**כמעט-פתרון.** `viewOf` מחזיר **מחרוזת אחת**, `render` שם אותה ב-`data-state`, וכל השאר בשורה נגזר ממנה. אין שם תנאי שני.

## י · "הלשונית פתוחה עשרים דקות ועדיין כתוב 'טרי'"

**דחיפה.** מה גורם לעמוד לצייר מחדש? ומה קרה בעשרים הדקות האלה?

**אסטרטגיה.** שום דבר. "טרי" נגזר מהזמן הנוכחי, אבל שום `setState` לא קרה, אז `render` לא רץ, אז השורה מציגה תשובה נכונה לשאלה שנשאלה לפני עשרים דקות.

**כמעט-פתרון.** `setInterval(render, 60000)`. שורה אחת, והיא זולה: `render` סינכרוני וקורא שני אובייקטים. **וזו בדיוק הסיבה שהטריות נגזרת ולא נשמרת** — אם היית שומר את התשובה, ציור מחדש לא היה עוזר.

## יא · "העמוד שולח בקשות בטעינה ואני לא רוצה"

**דחיפה.** מה קורא ל-`refreshCity` או ל-`refreshAll` בסוף `js/app.js`?

**אסטרטגיה.** הבוט אמור להיות `wire()` ואז `render()`, וזהו. **זו דרישה בברף**, לא העדפה: ציור ראשון שמחכה לשש בקשות הוא מלבן לבן על חיבור גרוע ושום דבר בלי חיבור.

**כמעט-פתרון.** ואם אתה עושה את חלק האתגר — הרענון האוטומטי בא **אחרי** הציור הראשון, ורק לערים שהן `stale`, ובלי להחליף את מה שמוצג בספינר. הסדר הוא כל התבנית.

### בדיקה שלוקחת שלושים שניות ותופסת חצי מהדברים כאן

Local Storage > Application < המפתח שלך. ערוך את הערך ביד: מחק סוגר, שים `null`, שים `[1,2,3]`, שנה `fetchAt` למספר קטן. רענן אחרי כל אחד. **העמוד חייב לעלות בכל אחד מהם**. אם הוא לא — יש לך את השורה המדויקת, ואת המחרוזת המדויקת שהפילה אותה.